객체지향 프로그래밍 고급

2장 : 클래스

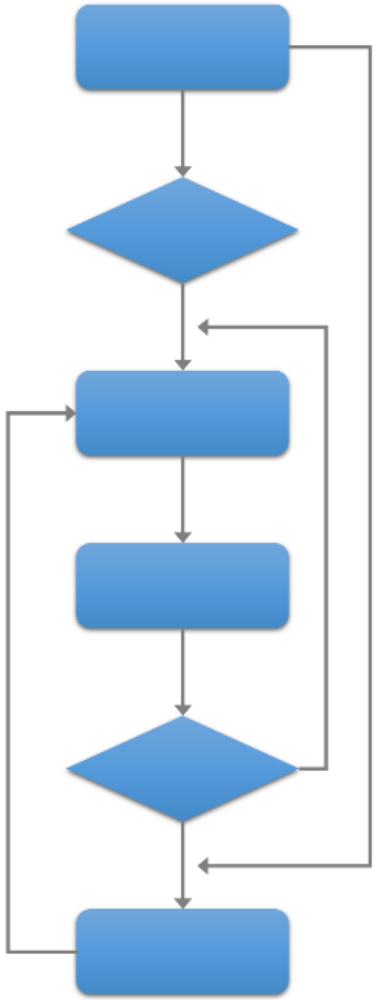
1. 객체지향 프로그래밍 이해

절차지향 프로그래밍

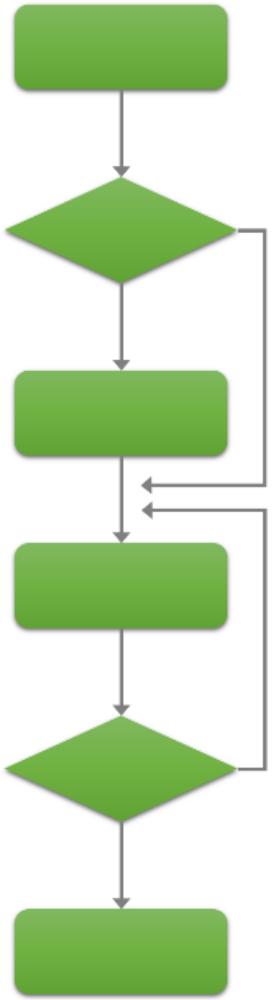
객체지향 프로그래밍

절차지향 프로그래밍

A. 나쁜 프로그램의 예



B. 좋은 프로그램의 예



출처 :현대마법의 IT 공.방 블로그

절차지향 프로그래밍

❑ 절차지향 프로그래밍의 특징

- 프로그램이 실행될 때 위에서 아래로의 절차, 순서에 맞게 실행됨
- 데이터와 처리동작을 별도로 구분하고 기능을 하나의 함수로 구현
- 예 – C언어

❑ 절차지향 프로그래밍의 단점

- 디버깅 및 프로그램의 유지보수가 어려움(실행순서가 바뀌면 동일한 결과를 보장하기 어려움 등..)
- 데이터와 처리 동작 사이의 관계가 밀접한 연관성을 갖지 못함
- 코드의 재사용이 어려움

객체지향 프로그래밍

❑ 객체(Object)란 무엇인가?

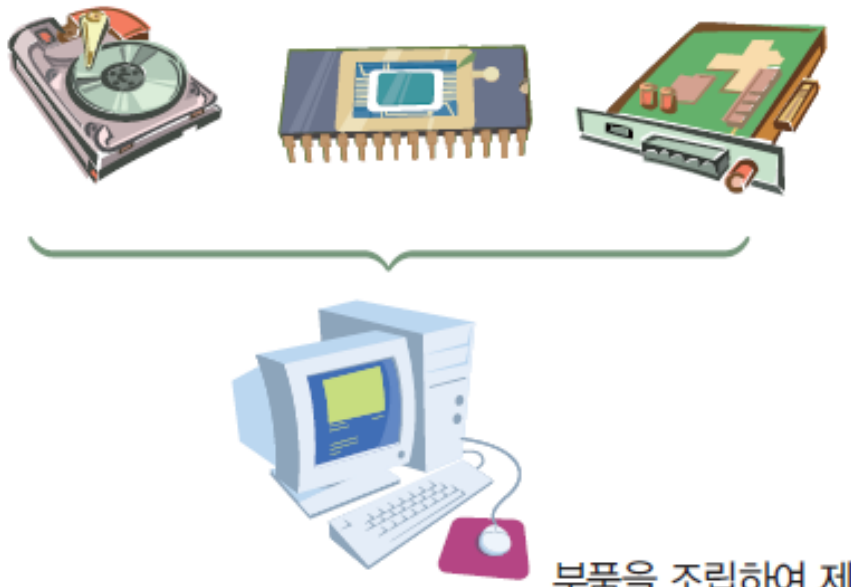
- 실세계에 존재하거나 생각할 수 있는 것을 객체(Object)라고 한다.
- 의자, 책상, 핸드폰같은 사물 뿐 아니라 강의, 수강신청같은 개념으로 존재하는 것도 모두 객체이다.
- 더 넓게 보면 인간이 생각하고 표현할 수 있는 모든 것이 객체이다.



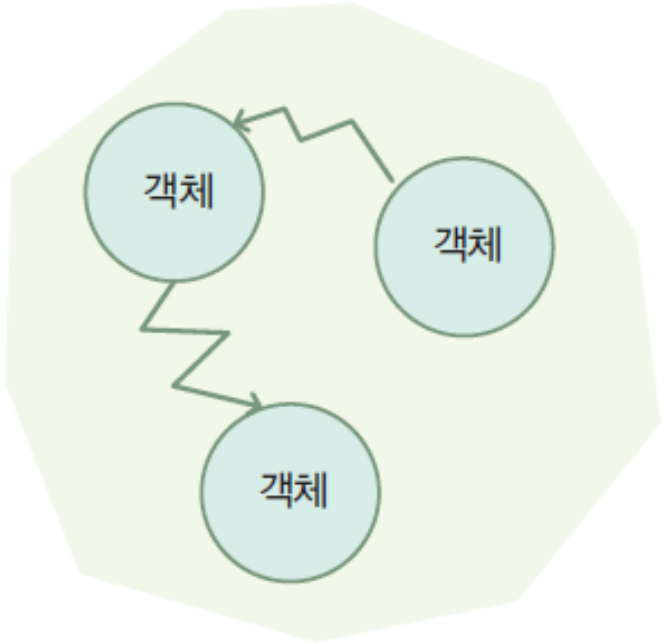
출처:쉽게 배우는 소프트웨어 공학

객체지향 프로그래밍

❑ 객체 지향으로 소프트웨어를 개발하는 것은 컴퓨터 하드웨어 부품을 구입하여서 컴퓨터를 조립하는 것과 비슷하다.



부품을 조립하여 제품을 만들듯이
객체들을 조립하여 소프트웨어를 만든다.



소프트웨어

객체지향 프로그래밍

❑ 장점

- 유지보수가 용이하다.
- 가독성이 높다.
- 신뢰성있는 소프트웨어를 작성할 수 있다.
- 코드를 재사용하기 쉽다.

❑ 단점

- 설계가 어렵다.
- 실행시간이 더 오래 걸릴 수 있다.

2 클래스

클래스(Class)와 객체(Object)

객체지향의 특성

클래스 설계의 시작

클래스와 객체

❑ 클래스란 무엇인가?

- C++에서 기본으로 제공하는 자료형으로 실세계의 모든 대상을 표현할 수 없으므로 사용자가 정의하여 새로운 자료형을 만들 수 있다.
- 객체에서 필요한 필수 정보들과 공통된 특성을 파악한 후 불필요한 특성은 제거하고 개념화(모델링)하여 공통된 특성을 모아 하나의 패키지 안에 넣은 것을 의미한다.
- 공통된 특성은 정보(데이터)와 기능(행동, 연산, 데이터조작)으로 구분한다.
- 설계도 같은 것

클래스와 객체

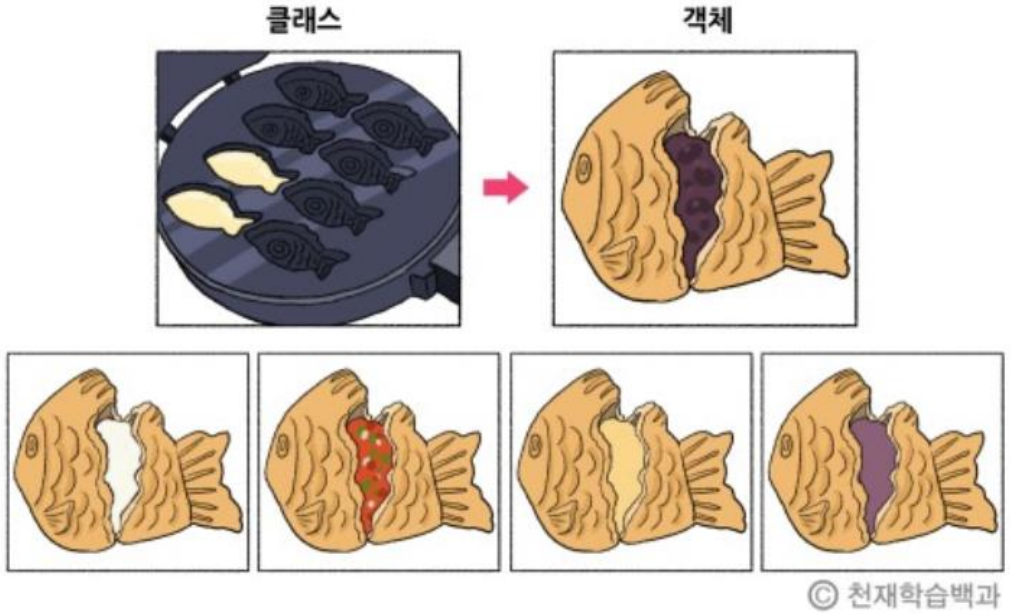
❑ 객체란 무엇인가?

- 클래스형의 변수이다.
- 객체는 사람, 동물, 차(car)처럼 현실 세계에 존재하거나 생각할 수 있는 모든 것을 말한다.
- 모든 객체는 데이터(속성, 특징, 상태)와 행위(행동, 동작, 기능)를 갖는다.

클래스와 객체

❑ 클래스와 객체의 관계

- 클래스로부터 만들어지는 객체를 그 클래스의 인스턴스(instance)라고 함.
- 클래스는 개념을 형상화한 것이고, 인스턴스는 현재 존재하는 것



클래스와 객체

❑ 예를 들면,

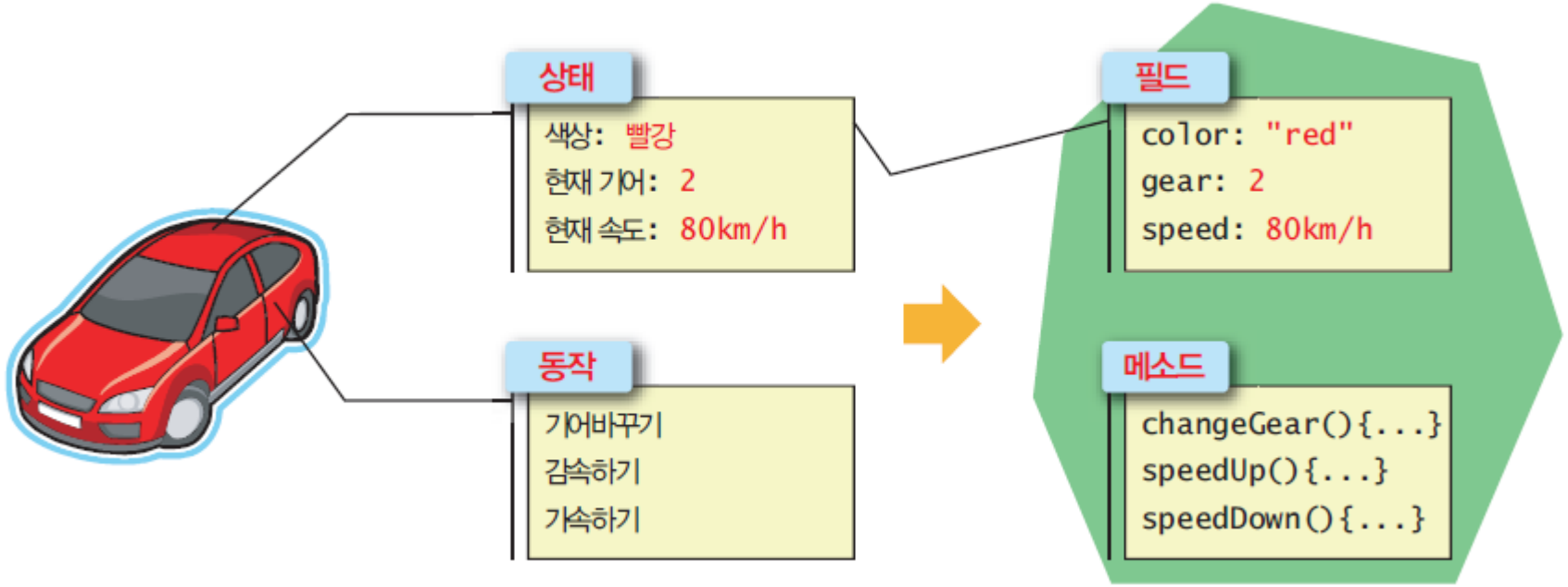
- 기아 자동차 공장에서 K7을 생산하기 위해 자동차 설계도를 만든다.
- 자동차의 색상, 옵션 등이 다양하므로 이를 하나의 설계도에 반영할 수 없다.
- 따라서 설계도에는 K7의 필수 기능을 표현하되, 변경되는 부분(색상, 옵션 등)은 객체에서 확정지을 수 있다.



클래스와 객체

❑ **모든 객체는 데이터와 기능을 갖는다.**

- 데이터 : 객체의 상태, 속성, 특징 (명사적 의미)
- 기능 : 객체가 취할 수 있는 행위, 행동, 동작 (동사적 의미)



객체지향의 특성

캡슐화(Encapsulation)

추상화(Abstraction)

상속성(Inheritance)

다형성(Polymorphism)

구성 요소	의미	목적
캡슐화	데이터와 기능을 하나로 묶고 정보 은닉	보안성, 유지보수
추상화	핵심 기능만 표현하고 세부 구현은 숨김	복잡성 감소
상속	기존 클래스의 기능을 물려받음	코드 재사용
다형성	같은 명령이 객체마다 다르게 동작	확장성, 유연성

객체지향의 특성 : 캡슐화

- 객체의 구성 요소들을 캡슐로 싸서 보호하고 볼 수 없게 하는 것

  - 예를 들면 캡슐에 든 약은 어떤 성분인지 보이지 않으며 외부로부터 보호된다.

- 사용자에게는 해당 객체의 기능과 사용법만 제공한다.



객체지향의 특성 : 캡슐화

- 전제 : “코감기는 항상 콧물, 재채기, 코막힘을 동반한다.”
- 코감기약을 클래스로 정의

  - ①콧물약() ②재채기약() ③코막힘약() 구성

  - 사용자는 세 개의 약 중 하나를 빠트리고 복용하면 안되며, 항상 ① ② ③의 순서로 약을 복용해야 한다.

- 코감기약을 캡슐화

  - 콧물약(), 재채기약(), 코막힘약()이 포함된 코감기약()을 구성

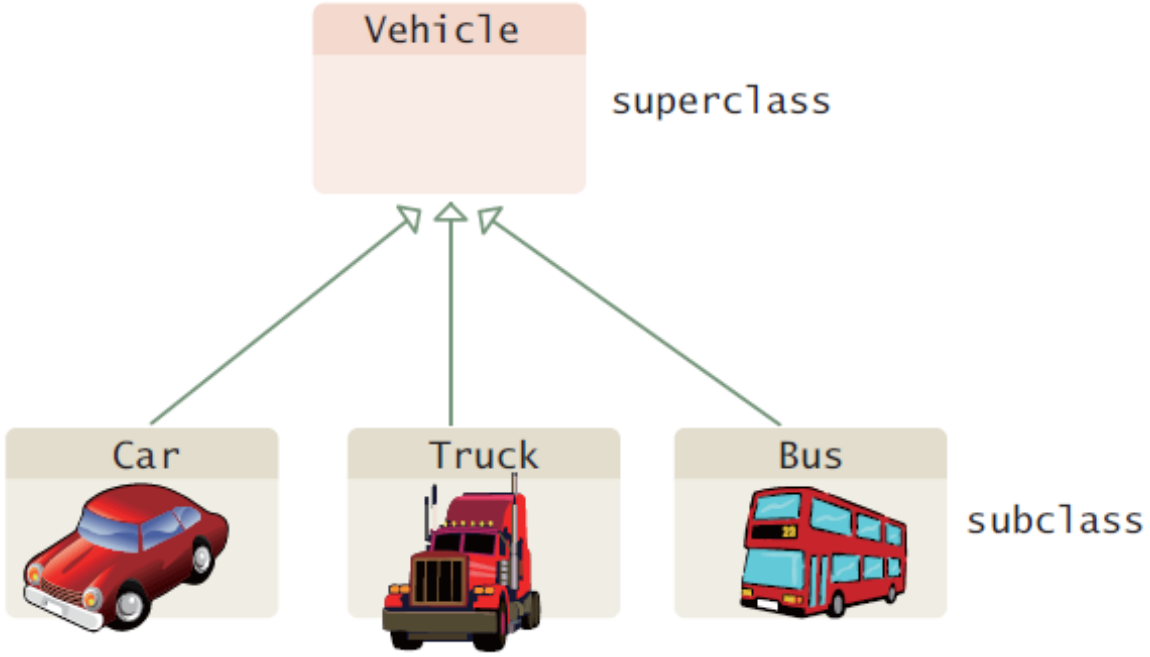
  - 사용자는 코감기약() 하나만 복용하면 되며 다른 약에 대해서는 신경쓰지 않아도 됨

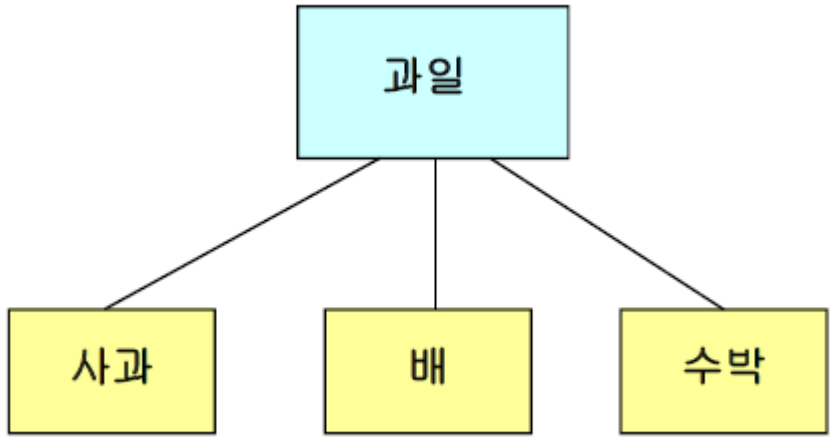
객체지향의 특성 : 정보은닉

- 불필요한 정보는 숨기고 중요한 정보만을 표현함
- 제한된 방법으로의 접근만 허용해서 잘못된 값이 저장되지 않도록 해야 하며, 실수가 쉽게 발견되도록 해야 한다.
- 내부 항목에 관한 정보는 감추고 인터페이스를 통해서만 메시지를 전달함으로서 다른 모듈을 변경하지 못하도록 한다.(private)

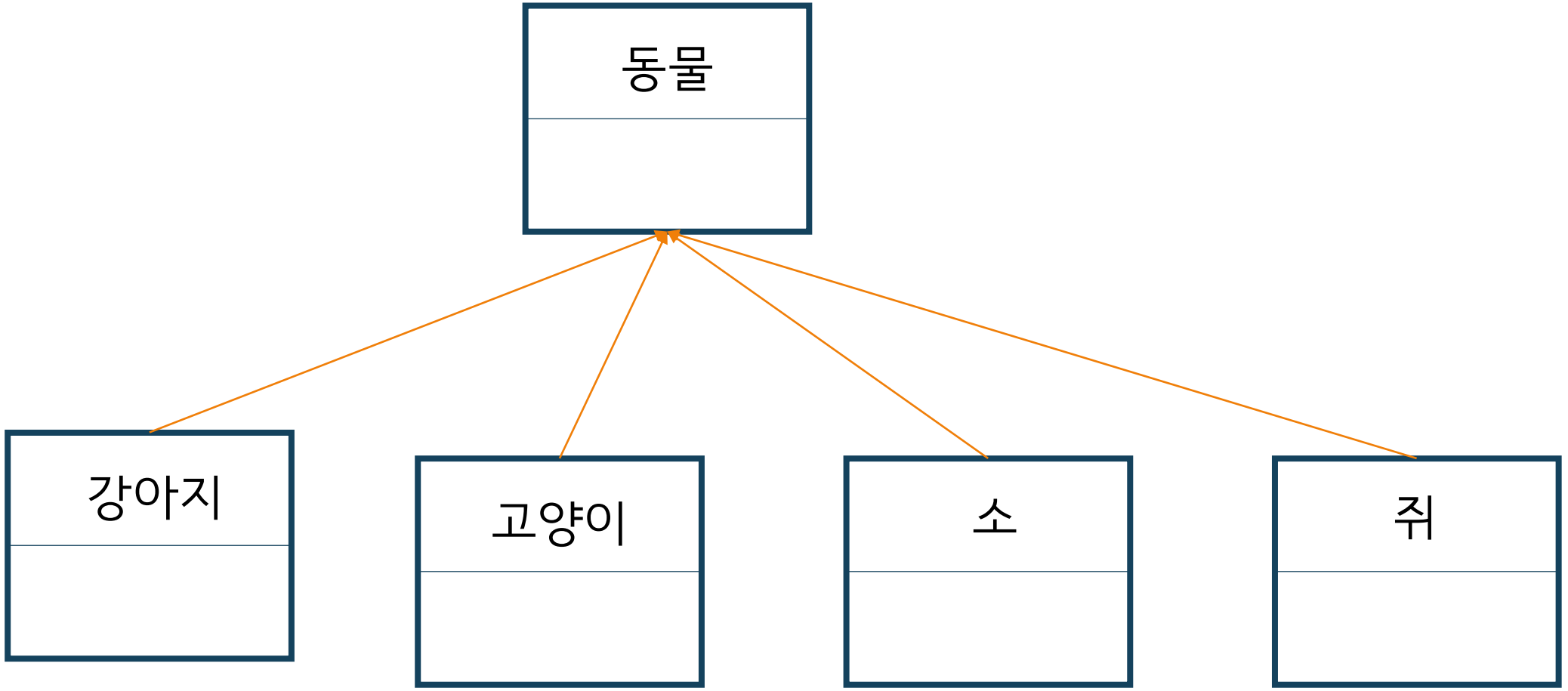
객체지향의 특성 : 상속성(inheritance)

- 이미 작성된 클래스(부모)를 이어받아 새로운 클래스(자식)를 생성하는 것
- 자식 클래스는 부모 클래스에서 정의된 속성(데이터 및 연산처리)을 그대로 물려받으면서 자신만의 속성을 더 첨가한 클래스를 정의



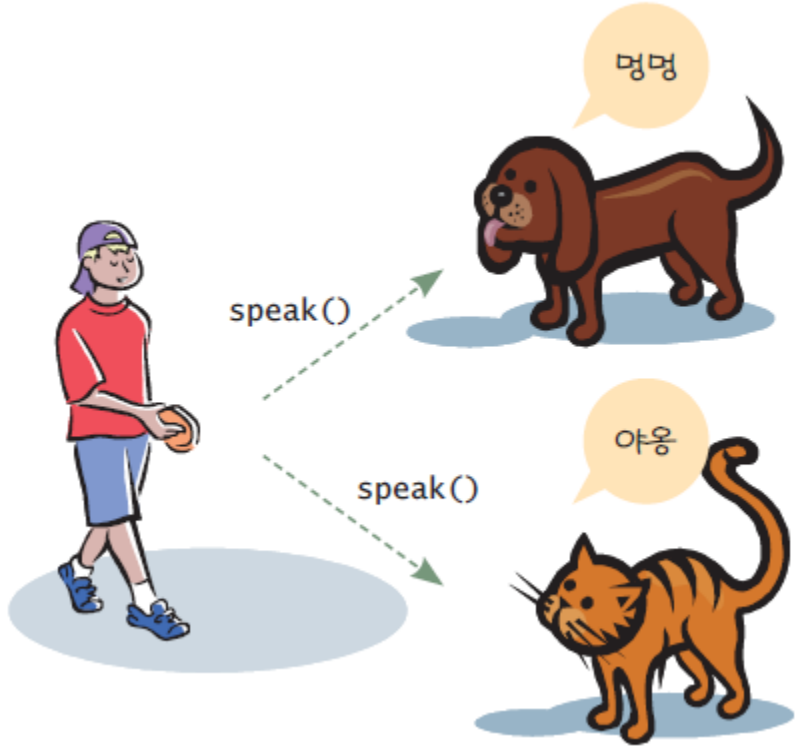


객체지향의 특성 : 상속성

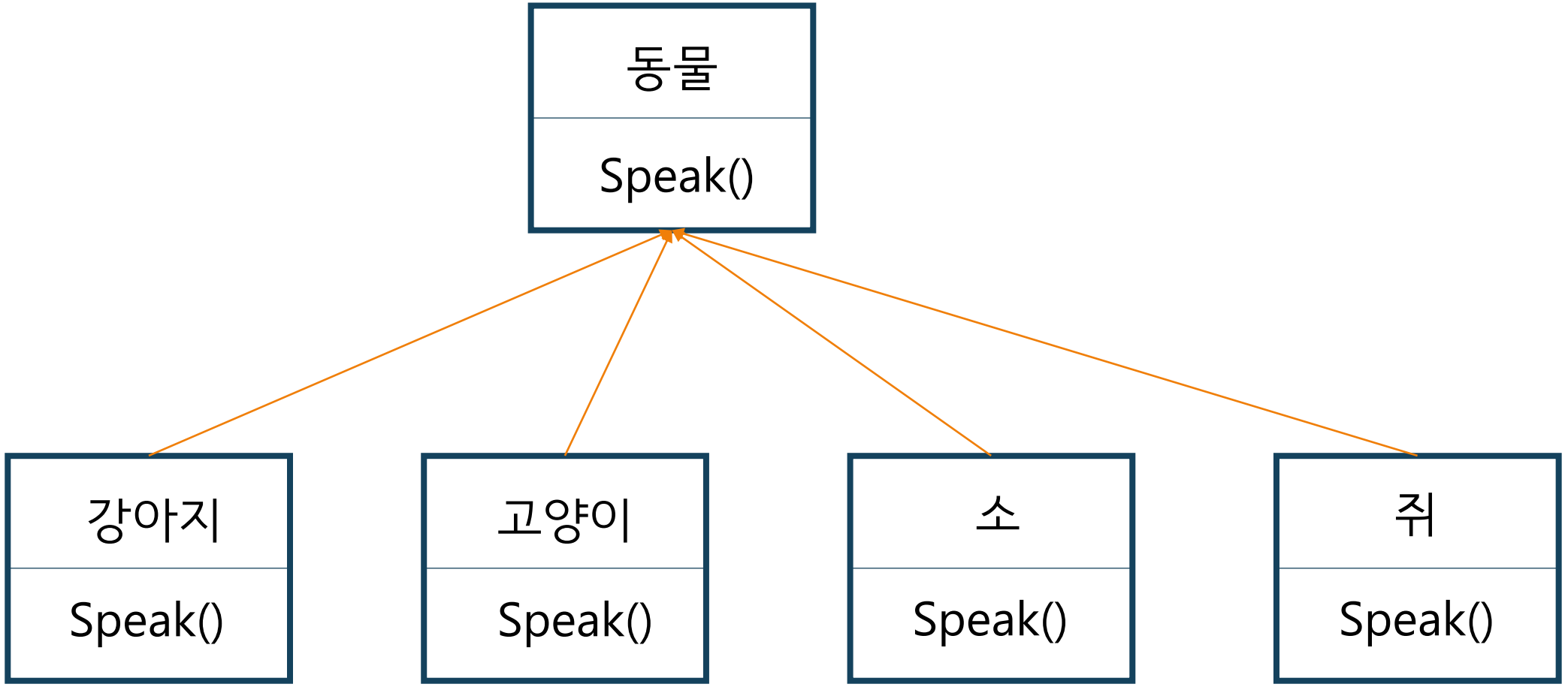


객체지향의 특성 : 다형성

- 서로 다른 클래스의 객체들이 같은 메시지에 대해 같은 의미의 응답을 하는 특성
- 각각 다른 클래스에 동일한 이름의 메서드를 정의해도 자료형으로 구별할 수 있음
- 개념적으로 동일한 작업을 하는 멤버 함수들에 똑같은 이름을 부여할 수 있으므로 코드가 더 간단해진다
- 함수 오버로딩, 연산자 오버로딩 등을 이용하여 다형성 지원



객체지향의 특성 : 다형성

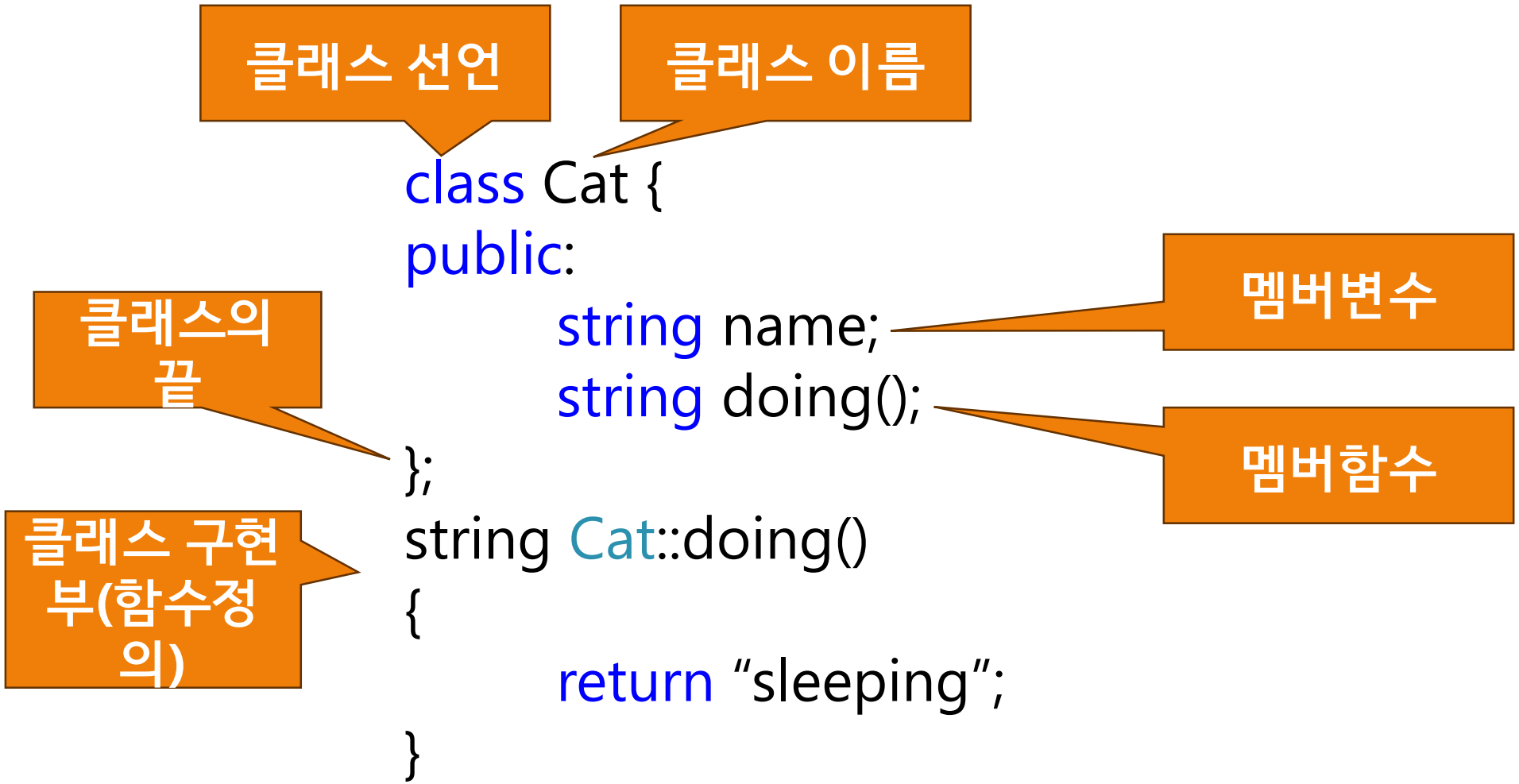


클래스(Class)와 객체(Object)

객체지향의 특성

클래스 설계의 시작

클래스 정의


클래스 선언
클래스 이름
class Cat {
public:
string name;
string doing();
};
string Cat::doing()
{
return “sleeping”;
}
멤버변수
멤버함수
클래스의 끝
클래스 구현부(함수정의)

클래스 구현의 두가지 방법

```
class Cat {
public:
        string name;
        string doing();
};
string Cat::doing()
{
        return “sleeping”;
}
```

```
class Cat {
public:
        string name;
        string doing()
        {
                return “sleeping”;
        }
};
```

클래스 정의



- 고양이를 설계해 보자.
- 고양이의 행동(함수)은
  - 먹는다.
  - 잔다.
  - 운다.
  - 고양이의 속성(데이터) 출력하기
- 고양이의 속성(변수)는
  - 이름
  - 나이
  - 품종

클래스 정의

```
class Cat
{
public:
    string name;
    string kind;
    int age;
    void showData();
    void eating(string food);
    void crying(string sound);
    void sleeping(int hours);
};

void Cat::showData()
{
    cout << "이름:" << name  << endl;
    cout << "종류:" << kind << endl;
    cout << "나이: " << age << endl;
}
```

```
void Cat::eating(string food)
{
    cout << name << "이 먹는 사료는 " << food
<<"입니다." << endl;
}

void Cat::crying(string sound)
{
    cout << name << "가 우는 소리는  "  << sound
<<"입니다.." << endl;
}

void Cat::sleeping(int hours)
{
    cout << name << "은 " << hours <<"시간을 잡는
다. " << endl;
}
```

객체 생성

클래스를 정의한 후 프로그램에서 사용하려면 클래스를 자료형으로 하는 변수를 선언해야 한다. 이 변수를 객체라고 한다.

클래스명 객체명;

Cat mycat;

Cat mycat, happy, marry;

객체 생성

```
class Cat
{
public:
    string name;
    string kind;
    int age;
    void showData();
    void eating(string food);
    void crying(string sound);
    void sleeping(int hours);
};

void Cat::showData()
{
    cout << "이름:" << name  << endl;
    cout << "종류:" << kind << endl;
    cout << "나이: " << age << endl;
}
```

```
int main()
{
    Cat mypet;
    mypet.name = "애옹이";
    mypet.kind = "코리안숏헤어";
    mypet.age = 5;
    mypet.ShowData();
    return 0;
}
```

객체 멤버 접근하기

클래스에 대한 객체를 정의한 후 클래스 외부에서 클래스의 멤버를 참조하여 사용하는 방법

객체명.멤버변수;
객체명.멤버함수();

Cat mycat;

mycat.name = “애옹이”;

mycat.showdata();

객체 멤버 접근하기

```
class Cat
{
public:
    string name;
    string kind;
    int age;
    void showData();
    void eating(string food);
    void crying(string sound);
    void sleeping(int hours);
};

void Cat::showData()
{
    cout << "이름:" << name  << endl;
    cout << "종류:" << kind << endl;
    cout << "나이: " << age << endl;
}
```

```
int main()
{
    Cat mypet;
    mypet.name = "애옹이";
    mypet.kind = "코리안숏헤어";
    mypet.age = 3;
    mypet.ShowData();
    return 0;
}
```

[예제1] 학생정보 클래스 만들기

학생의 이름과 학번을 갖는 클래스를 선언하고 자신의 데이터를 할당한 후, 출력하시오.

속성
 -이름:
 -학번

기능
 -정보 출력

```
class Student
string name;
string id;

void dataShow() // 출력하는 함수
```

접근제어 지시자

public 어디서든 접근허용

protected 상속관계에 놓여 있을 때 접근허용

private 클래스 내(클래스 내에 정의된 함수)에서만 접근허용

접근제어 지시자를 선언하지 않으면 자동으로 클래스는 private, 구조체는 public으로 선언된다.

접근제어 지시자

```
class Cat
{
public:
    string name;
    string kind;
    int age;
    void showData();
    void eating(string food);
    void crying(string sound);
    void sleeping(int hours);
};

void Cat::showData()
{
    cout << "이름:" << name  << endl;
    cout << "종류:" << kind << endl;
    cout << "나이: " << age << endl;
}
```

```
int main()
{
    Cat mypet;
    mypet.name = "애옹이";
    mypet.kind = "코리안숏헤어";
    mypet.age = 5;
    mypet.ShowData();
    return 0;
}
```

접근제어 지시자

- 접근제어 지시자를 “public”으로 했을 경우의 문제점

```
mypet.age = 5;
```

⇒ 클래스 외부에서 누구나 접근하여 데이터를 변경할 수 있다.

⇒ 멤버 변수는 private으로 선언하여 **정보은닉(비공개)**하는 것이 좋다.

접근제어 지시자

```
class Cat
{
private:
    string name;
    string kind;
    int age;
public:
    void showData();
    void eating(string food);
    void crying(string sound);
    void sleeping(int hours);
};

void Cat::showData()
{
    cout << "이름:" << name  << endl;
    cout << "종류:" << kind << endl;
    cout << "나이: " << age << endl;
}
```

```
int main()
{
    Cat mypet;
    mypet.name = "애옹이";
    mypet.kind = "코리안숏헤어";
    mypet.age = 3;
    mypet.ShowData();
    return 0;
}
```

사용 불가

접근제어 지시자

```
class Cat
{
private:
    string name;
    string kind;
    int age;
public:
    void showData();
    void setName(string str) { name =
str; }
    void setAge(int a) { age = a;}
};
void Cat::ShowData()
{
    cout << “이름:” << name  << endl;
    cout << “종류:” << kind << endl;
```

접근제어 지시자를 private으로 하면,

- Cat class를 제외한 어떤 곳에서도 멤버변수 name, kind, age를 직접 변경할 수 없다. (main함수에서도 변경불가)
- 데이터설정/변경을 하기 위한 별도의 public 함수가 필요(생성자 또는 setAge(), SetName() 등)

정보은닉(Information Hiding)

두 프로그램의 차이점은 ?

```
class Point
{
   public:
            int xpos;
            int ypos;
            void ShowData() {
               cout << xpos << " , " << ypos<<endl;
            }
};

int main()
{
   Point pos1 = { -2, 4 };
   . . .
   return 0;
}
```

```
class Point
{
   private:
            int xpos;
            int ypos;
   public:
            bool InitMembers(int x, int y) {
                if (x < 0 || y < 0) return false;
                xpos = x;
                ypos = y;
                return true;
            }
            void ShowData() {
               cout << xpos << " , " << ypos<<endl;
            }
};
int main()
{
   Point pos1;
   if (!pos1.InitMembers(-2, 4)) exit(1);
   . . .
   return 0;
}
```

[예제2] 학생정보 클래스 만들기

학생의 이름과 학번을 갖는 클래스를 선언하고 자신의 데이터를 입력받는 함수와 출력하는 함수를 추가하라.

속성	class Student
-이름:	string name;
-학번	string id;
기능	
-데이터 입력	void SetData(string name, string id);
-데이터 출력	void dataShow() // 출력하는 함수

[예제3] box 클래스 설계



가로, 세로, 높이가 있는 직육면체의 속성과 부피를 구하는 함수를 정의한 box 클래스를 설계하고, 각각의 데이터가 아래와 같을 때 직육면체의 부피를 출력하라.

가로, 세로, 높이 : 10, 5, 9

```
int width;
int length;
int height;

int calVolume(); // 부피 : 가로x세로x높이
```

[예제4] 학생 클래스 설계

아래와 같이 학생에 대한 정보를 갖는 클래스를 설계하되, 데이터를 입력받는 함수와 출력하는 함수를 정의하고 데이터를 출력하시오.

속성	데이터 타입	데이터
학번	string	1234564
이름	string	아이유
국어	int	98
영어	int	100

```
class Student
{
private:
    string stdid;
    string name;
    int   Kor;
    int   Eng;
public:
    void setInfo(string stdid, string name);
    void setKorean(int k);
    void setEnglish(int e);
    void showData();
}
```

```
class Student
{
private:
    string stdid;
    string name;
    int   Kor;
    int   Eng;
public:
    void setMember(string id, string n);
    void setKorean(int k) { Kor = k; }
    void setEnglish(int e) { Eng = e; }
    void showData();};
};
void Student::showData()
{
   cout << "학번 : " << id << endl;
   cout << "이름 : " << name << endl;
   cout << "국어 : " << Kor << endl;
   cout  << "영어 : " << Eng << endl;
}
```

```
void Student::setMember(string id, string n)
{
    stdid = id;
    name = n;
}


int main()
{
    Student me;
    me.setMember("0821034", "아이유");
    me.setKorean(98);
    me.setEnglish(100);
    me.showData();
    return 0;
}
```

[예제5] 프린터 클래스 설계

Printer 클래스를 정의하라. Printer는 문자열을 입력받아 저장하거나 출력하는 기능을 갖고 있다. 이 클래스의 속성과 기능은 다음과 같다.

- 속성(데이터) : 문자열
- 기능 : (1)문자열 저장 (2)문자열 출력

입력되는 문자열은 다음과 같다.

Hello World!

```
class Printer
{
private:
    string str;
public:
    void SaveString(string s);
    void PrintString();
};
void Printer::SaveString(string s)
{
    str = s;
}
void Printer::PrintString()
{
    cout << str << endl;
}
```

```
int main()
{
    Printer p;

    p.SaveString("Hello World!");
    p.PrintString();

    return 0;
}
```

[예제6] 포인트 클래스 설계

2차원 그래프에서 x, y축의 좌표를 나타내는 Point 클래스를 정의하시오.
x, y 좌표를 저장할 수 있는 setPos()함수와 좌표를 출력하는 Show() 함수도 정의하시오.